Kerberos for Infrastructure Engineers: Production-Grade Mastery

Section 1: The 80/20 Core (Hiring-Critical)

1. Credential Cache Lifecycle: The Silent Killer of Service Uptime

Why hiring managers care: Most Kerberos outages aren't KDC failures—they're credential cache mismanagement. If you can't explain cache types, locations, and renewal behavior, you'll cause SEV-1s.

The Core Reality:

```
# Three cache types engineers confuse:
FILE:/tmp/krb5cc_1000          # Default per-UID file cache
KEYRING:persistent:1000        # Kernel keyring (survives logout)
KCM:                           # Kerberos Credential Manager (daemon)
```

Library Behavior That Matters:
- `krb5_cc_default()` searches `KRB5CCNAME` env → `/tmp/krb5cc_$(id -u)` → first valid cache
- Credential caches are **NOT locked** by default—race conditions when multiple processes write
- Cache files contain the **entire TGT** (ticket-granting ticket) in plaintext → steal the file = impersonate user
- `kinit` overwrites the default cache; `kinit -c` isolates but requires setting `KRB5CCNAME` everywhere

Production Gotcha:

```
# Service startup script:
kinit -kt /etc/myapp.keytab myapp/host.example.com
# Runs as root, writes to /tmp/krb5cc_0

# Application code:
# Runs as 'myapp' user, looks for /tmp/krb5cc_1001
# → Authentication fails silently, falls back to unauthenticated mode
```

What breaks: Containerized services where `/tmp` isn't shared between init and app containers. Cron jobs that don't set `KRB5CCNAME`. Systemd services that run `kinit` in `ExecStartPre` but don't export the cache location.

Hiring signal: Can you explain why `klist` shows valid tickets but `curl --negotiate` fails? (Answer: wrong cache for the process UID)

---

2. Keytabs: The Password-Free Authentication Trap

Why hiring managers care: Keytabs are how 99% of services authenticate in production. Misunderstanding them causes privilege escalation and credential leakage.

The Core Reality:
A keytab is a **local copy of the service's long-term key** (derived from its "password" in the KDC database). It lets processes authenticate without human interaction.

```
# Generate keytab (AD example):
ktpass -princ HTTP/webserver.corp.com@CORP.COM -mapuser CORP\webserver$ \
       -crypto AES256-SHA1 -ptype KRB5_NT_PRINCIPAL -out webserver.keytab

# Use it:
kinit -kt webserver.keytab HTTP/webserver.corp.com
```

Library Behavior That Breaks Systems:
1. **Keytabs don't expire, but the keys inside them do**
- When you reset a service account password in AD → all keytabs using that account's key instantly break
- No warning, no error log until authentication is attempted

1. **Multiple keys per principal are supported but poorly documented**
   `bash klist -ke /etc/krb5.keytab # 1 HTTP/webserver.corp.com@CORP.COM (aes256-cts-hmac-sha1-96) # 2 HTTP/webserver.corp.com@CORP.COM (aes128-cts-hmac-sha1-96) # 3 HTTP/webserver.corp.com@CORP.COM (arcfour-hmac)`
   - Library tries newest key first (`kvno` number), falls back
   - But if encryption types are disabled in KDC policy → silent failure

2. **Keytab permissions are a privilege escalation vector**
   ```bash
   # Common mistake:
   -rw-r--r-- 1 root root 1234 /etc/app.keytab
   ```

Any user can now:
kinit -kt /etc/app.keytab app/service@REALM
→ Impersonate the service principal
`` - Correct: chmod 600`, owned by service user

Production Gotcha:

```
# Engineer rotates service account password in AD
# Forgets to regenerate keytab
# Service uses cached TGT for 10 hours (default lifetime)
# TGT expires → service tries to get new TGT with keytab → FAILS
# → Silent degradation until tickets expire
```

Hiring signal: Can you explain why a service works fine for hours after a password reset, then suddenly fails? (Answer: cached TGT vs. keytab key mismatch)

---

3. Clock Skew: The 5-Minute Rule That Isn't

Why hiring managers care: Time synchronization is assumed infrastructure. When it breaks, Kerberos is the canary. You need to diagnose this in minutes, not hours.

The Core Reality:
Kerberos tickets have timestamps to prevent replay attacks. By default, libraries enforce:

```
// libkrb5 source (simplified):
if (abs(ticket_time - local_time) > clockskew) {
    return KRB5KRB_AP_ERR_SKEW;
}
```

Default `clockskew` in `/etc/krb5.conf`:

```
[libdefaults]
    clockskew = 300  # 5 minutes (MIT default)
```

What Actually Happens:
- **Server-side check:** KDC validates request timestamp when issuing TGT
- **Client-side check:** Application server validates ticket timestamp when accepting service ticket
- **Two clocks matter:** Client-to-KDC and Client-to-AppServer

Production Gotcha:

```
Client (12:00:00) → KDC (12:00:02) → Issues TGT valid 12:00:00-22:00:00
Client (12:01:00) → AppServer (12:07:00) → Rejects ticket (7 min skew > 5 min)
```

- NTP daemon crashes on app server
- VM clock drift in cloud (especially after snapshots/migrations)
- Leap second handling in containers

Library Behavior:

```
# Error you see:
$ kinit user@REALM
kinit: krb5_get_init_creds: Clock skew too great

# Actual problem:
$ date && ssh kdc.example.com date
Wed Feb  4 14:23:17 MST 2026
Wed Feb  4 14:29:42 MST 2026  # 6+ min difference
```

The Hiring Trap: Junior engineers increase `clockskew` to 3600 (1 hour). This "fixes" the symptom but breaks security assumptions:
- Replay window increases
- Ticket reuse attacks become viable
- Compliance violations (PCI-DSS requires time sync)

Correct fix: Fix NTP/chrony, don't mask the problem.

Hiring signal: Can you explain why changing `clockskew` to 1 hour is a security issue? (Answer: replay attack window, violates defense-in-depth)

---

4. Service Principal Names (SPNs): The Hidden Namespace

Why hiring managers care: SPN misconfiguration is the #1 cause of "works on my laptop, fails in prod" Kerberos failures. This is pure operational maturity.

The Core Reality:
When a client wants to talk to a service, it requests a service ticket for a **specific SPN**:

```
service/hostname@REALM
```

Library Behavior (Reverse DNS Lookup):

```
// What happens when you do:
curl --negotiate https://webserver.corp.com/api

// Library does:
1. Parse URL → hostname = "webserver.corp.com"
2. getaddrinfo(webserver.corp.com) → 10.1.2.3
3. getnameinfo(10.1.2.3, NI_NAMEREQD) → Reverse DNS → "webserver.prod.corp.com"
4. Request ticket for HTTP/webserver.prod.corp.com@CORP.COM
5. If KDC doesn't have that SPN → NO_MATCH error
```

Production Gotcha #1: DNS Mismatch

```
# Service registered as:
setspn -A HTTP/webserver.corp.com CORP\webserver$

# But reverse DNS returns:
$ dig -x 10.1.2.3 +short
webserver.prod.corp.com.

# → Client requests HTTP/webserver.prod.corp.com
# → KDC has HTTP/webserver.corp.com
# → FAIL: Server not found in Kerberos database
```

Production Gotcha #2: Load Balancers

```
# VIP: lb.corp.com → 10.1.2.100 (LB)
#   → 10.1.2.101 (webserver1)
#   → 10.1.2.102 (webserver2)

# Client requests: HTTP/lb.corp.com
# But keytab on webserver1 has: HTTP/webserver1.corp.com
# → Ticket doesn't match server principal → FAIL
```

The Fix (Host-Based Services):

```
# /etc/krb5.conf
[libdefaults]
    rdns = false              # Disable reverse DNS
    ignore_acceptor_hostname = true  # Accept any hostname in ticket
```

Risk: Disabling these checks weakens security (enables DNS spoofing attacks).

The Right Fix: Register all required SPNs:

```
setspn -A HTTP/lb.corp.com CORP\webserver1$
setspn -A HTTP/lb.corp.com CORP\webserver2$
```

Hiring signal: Can you debug this from packet captures? (Look for `KRB_AP_ERR_NOT_US` or `Server not found` in KDC logs)

---

5. Ticket Renewal vs. Ticket Lifetime: Long-Running Processes

Why hiring managers care: Services that run for days/weeks must handle ticket renewal. Failure = silent auth degradation in production.

The Core Reality:
Every Kerberos ticket has two lifetimes:

```
$ klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: user@CORP.COM

Valid starting     Expires            Service principal
02/04/26 14:00:00  02/05/26 00:00:00  krbtgt/CORP.COM@CORP.COM  # TGT
    renew until 02/11/26 14:00:00  # ← THIS IS KEY
```

- **Lifetime:** How long the ticket is valid (default: 10 hours)
- **Renewable lifetime:** How long you can renew it without re-authenticating (default: 7 days)

Library Behavior:

```
// Automatic renewal (NOT default in MIT Kerberos):
krb5_get_renewed_creds(context, &creds, principal, ccache, NULL);

// Manual renewal:
kinit -R  # Only works if renewable lifetime hasn't expired
```

Production Gotcha:

```
# Python service using gssapi:
import gssapi
```

```
# Initial auth with keytab:
cred = gssapi.Credentials(
    name=gssapi.Name('service/host@REALM'),
    usage='initiate',
    store={'keytab': '/etc/service.keytab'}
)

# Service runs for 30 days
# Ticket expires after 10 hours
# gssapi library does NOT auto-renew
# → Authentication fails silently after 10 hours
```

The Fix (Library-Specific):

```
# Java (automatic renewal):
System.setProperty("sun.security.krb5.principal", "service/host@REALM");
System.setProperty("javax.security.auth.useSubjectCredsOnly", "false");
// Java GSS-API renews automatically

# Python (manual renewal required):
def renew_ticket():
    subprocess.run(['kinit', '-R'], check=True)

# Run in background thread every 8 hours
```

MIT Kerberos k5start Solution:

```
# Wrapper that auto-renews:
k5start -f /etc/service.keytab -U -K 60 -- /usr/bin/myservice
# -K 60: Check every 60 min, renew if <10 min left
```

Hiring signal: Can you explain why a service works for 10 hours then fails, even with a valid keytab? (Answer: ticket expiry without renewal)

---

6. Delegation: The Trust Boundary Nightmare

Why hiring managers care: Delegation is how multi-tier apps work (web → API → database). Misconfiguring it causes security incidents or breaks SSO.

The Core Reality:
User authenticates to WebServer → WebServer needs to call APIServer **as the user** (not as WebServer).

Three delegation types:
1. **No delegation:** WebServer can't get tickets for the user (default, breaks multi-tier)
2. **Unconstrained delegation:** WebServer can impersonate user to ANY service (dangerous)
3. **Constrained delegation:** WebServer can impersonate user to SPECIFIC services (correct)

Library Behavior:

```
# Request forwardable TGT:
kinit -f user@REALM

$ klist
Flags: FIA  # F = Forwardable, I = Initial, A = Forwardable
```

Production Gotcha (AD):

```
# Enable unconstrained delegation (DANGER):
Set-ADComputer webserver -TrustedForDelegation $true

# What this allows:
# 1. User logs into webserver
# 2. Webserver caches user's TGT
# 3. Attacker compromises webserver
# 4. Attacker uses cached TGT to access ANY service as user
# → This is how Golden Ticket attacks start
```

Constrained Delegation (S4U2Self + S4U2Proxy):

```
# Allow webserver to impersonate users ONLY to specific services:
Set-ADComputer webserver -Add @{
    'msDS-AllowedToDelegateTo'=@(
        'http/apiserver.corp.com',
        'mssql/dbserver.corp.com'
    )
}
```

Library Code (Java):

```
// Request service ticket with delegation:
GSSContext context = manager.createContext(
    serviceName,
    GSSManager.DEFAULT_MECH,
    delegCred,  // User's delegated credential
    GSSContext.DEFAULT_LIFETIME
);
context.requestMutualAuth(true);
context.requestCredDeleg(true);  // ← Request delegation
```

Hiring signal: Can you explain the security difference between unconstrained and constrained delegation? (Answer: blast radius of compromise)

---

7. KDC Location: The DNS SRV Trap

Why hiring managers care: KDC discovery failures cause total auth outages. You need to understand the fallback chain.

The Core Reality:
Libraries find KDCs in this order:
1. `kdc =` in `/etc/krb5.conf` (explicit)
2. DNS SRV records: `_kerberos._udp.REALM`
3. DNS A record for `REALM` (legacy fallback)

Library Behavior:

```
# DNS SRV lookup:
$ dig _kerberos._tcp.corp.com SRV +short
0 100 88 kdc1.corp.com.
10 50 88 kdc2.corp.com.
10 50 88 kdc3.corp.com.

# Priority 0 = primary (kdc1)
# Priority 10 = secondary (kdc2, kdc3 load-balanced 50/50)
```

Production Gotcha:

```
# /etc/krb5.conf
[realms]
    CORP.COM = {
        kdc = kdc1.corp.com  # ← Hardcoded, no failover!
```

```
    }

# kdc1 goes down → all auth fails
# Even though kdc2/kdc3 are healthy
```

The Fix:

```
[realms]
    CORP.COM = {
        kdc = kdc1.corp.com
        kdc = kdc2.corp.com
        kdc = kdc3.corp.com
        # OR: rely on DNS SRV (remove kdc= lines)
    }
```

Active Directory Specificity:

```
# AD DCs register themselves in DNS:
$ dig _kerberos._tcp.dc._msdcs.corp.com SRV +short
# Updates automatically when DCs added/removed
```

UDP vs. TCP:
- Libraries try UDP first (port 88)
- If response > 1400 bytes (lots of groups/claims) → automatic TCP retry
- Some firewalls block UDP 88 → silent 5-second timeout per KDC → slow auth

Hiring signal: Can you explain why auth takes 15 seconds after a KDC failure? (Answer: UDP timeout on dead KDC before failover)

---

8. The "Fails Silently" Catalog

Why hiring managers care: Kerberos errors are cryptic. Knowing where things fail silently separates senior from junior engineers.

Common Silent Failures:

A) Wrong credential cache, authentication succeeds elsewhere:

```
# Process 1 (UID 0): kinit -kt /etc/app.keytab
# → Writes to /tmp/krb5cc_0
```

```
# Process 2 (UID 1001): attempts Kerberos auth
# → Looks for /tmp/krb5cc_1001, not found
# → Falls back to unauthenticated or fails silently
```

B) Expired keytab key (after password reset):

```
# Symptom: Service works for hours, then fails
# Reason: TGT cached, expires, renewal attempt uses stale keytab key
# Error: "Preauthentication failed" (but only in KDC logs)
```

C) Encryption type mismatch:

```
# KDC policy: AES256 only
# Keytab contains: RC4-HMAC only
# Symptom: "Preauthentication failed" or "No supported encryption types"
# Often happens during AD → MIT Kerberos migrations
```

D) Missing reverse DNS:

```
# Client requests: HTTP/webserver.corp.com
# Reverse DNS returns: (nothing)
# Client falls back to: HTTP/10.1.2.3 (IP address SPN)
# Server has: HTTP/webserver.corp.com
# → Ticket mismatch, fails with "Server not found"
```

E) Firewall blocks UDP 88, TCP works:

```
# Library tries UDP → 5s timeout
# Retries TCP → works
# User sees: 5-second delay every auth
# Logs: Nothing (timeout is expected behavior)
```

Hiring signal: Given a symptom ("auth works sometimes, fails other times"), can you generate a hypothesis list? (Answer: credential cache race, DNS caching, KDC failover)

Section 2: Genius-Level Understanding (Library-Centric)

Mental Model: Kerberos as a Distributed Capability System

Stop thinking of Kerberos as "authentication." It's a **capability token distribution system** with time-bounded trust delegation.

The Analogy:
- **TGT** = Master key to a token mint (capability to request capabilities)
- **Service Ticket** = Unforgeable bearer token with embedded proof of identity
- **KDC** = Trusted third party that mints tokens, but doesn't participate in actual authorization
- **Ticket Lifetime** = Capability expiration (prevents infinite replay)

Why this matters for library behavior:

Traditional authn: "Prove who you are to the server"

```
Client → Server: "I'm Alice" + password
Server: Validates password, grants access
```

Kerberos: "Present unforgeable proof you convinced the KDC you're Alice"

```
Client → KDC: "I'm Alice" + (password or keytab)
KDC → Client: TGT (encrypted ticket, usable ONLY by Alice)
Client → KDC: "Give me token for DBServer" + TGT
KDC → Client: Service Ticket (encrypted for DBServer, contains Alice's identity)
Client → DBServer: Service Ticket + Authenticator
DBServer: Decrypts ticket (only DBServer can), validates, grants access
```

Critical Insight: The service ticket is **encrypted with the service's key**. The client can't read or forge it. The server decrypts it and sees:
- Who issued it (KDC signature)
- Who it's for (client identity)
- When it expires
- What privileges/groups the client has

Library Reality:

```
// What's in a service ticket (simplified):
struct krb5_ticket {
    krb5_principal client;        // "alice@CORP.COM"
    krb5_principal server;        // "HTTP/webserver.corp.com@CORP.COM"
```

```
    krb5_timestamp authtime;      // When user initially authenticated
    krb5_timestamp starttime;     // When ticket becomes valid
    krb5_timestamp endtime;       // When ticket expires
    krb5_flags flags;             // FORWARDABLE, PROXIABLE, RENEWABLE, etc.
    krb5_authdata *authorization; // PAC (Windows), POSIX groups, etc.
    krb5_encrypted_data enc_part; // Encrypted with service's key
};
```

The Capability System Perspective:
- **Ambient authority is banned:** You can't say "I'm Alice, trust me." You must present a ticket.
- **Tickets are bearer tokens:** Possession = authority (hence cache file security matters)
- **No revocation:** Once issued, a ticket is valid until expiry (hence short lifetimes)
- **Delegation = capability delegation:** Forwarding a TGT is literally giving someone your "mint tokens" capability

Production Implication:

```
# Stolen credential cache = stolen capabilities:
$ cp /tmp/krb5cc_1000 /tmp/stolen
$ KRB5CCNAME=/tmp/stolen klist
# → I can now impersonate that user until ticket expires
# → No way to revoke except wait for expiry

# This is why:
# 1. Credential caches must be root-only or UID-isolated
# 2. Short ticket lifetimes are critical
# 3. KEYRING caches are better (kernel-isolated per session)
```

The Trust Graph: Multi-Realm and Cross-Domain

Production Scenario: Acquisition integrates Company A (CORP-A.COM) and Company B (CORP-B.COM) realms.

Library Behavior (Cross-Realm Authentication):

```
# User in CORP-A.COM wants to access service in CORP-B.COM
alice@CORP-A.COM → HTTP/webserver.corp-b.com@CORP-B.COM

# Path:
1. Alice gets TGT from CORP-A.COM KDC
2. Alice requests cross-realm TGT for CORP-B.COM
   → KDC returns krbtgt/CORP-B.COM@CORP-A.COM
```

```
3. Alice presents cross-realm TGT to CORP-B.COM KDC
4. CORP-B.COM KDC issues service ticket for HTTP/webserver.corp-b.com
```

The Trust Graph:

```
CORP-A.COM ←→ CORP-B.COM  (bidirectional trust)
CORP-A.COM → VENDOR.COM   (one-way trust, CORP-A trusts VENDOR)
```

krb5.conf Configuration:

```
[realms]
    CORP-A.COM = {
        kdc = kdc-a.corp-a.com
    }
    CORP-B.COM = {
        kdc = kdc-b.corp-b.com
    }

[domain_realm]
    .corp-a.com = CORP-A.COM
    .corp-b.com = CORP-B.COM

[capaths]
    CORP-A.COM = {
        CORP-B.COM = .  # Direct trust
    }
```

Production Gotcha (Transitive Trust in AD):

```
Forest Root: CORP.COM
  → Child: US.CORP.COM
  → Child: EU.CORP.COM

# AD automatically creates trust path:
alice@US.CORP.COM → service@EU.CORP.COM
# Path: US.CORP.COM → CORP.COM → EU.CORP.COM

# BUT: Ticket includes claims/groups from intermediate realms
# → Bloated tickets (>64KB) → TCP fallback → slow auth
```

Security Implication:

```
# If you compromise ANY realm in the trust graph,
# you can request tickets for ANY other realm
# → Lateral movement across org boundaries

# This is why:
# 1. Selective authentication (SID filtering in AD)
# 2. Trust relationships are high-security boundaries
# 3. Monitoring cross-realm TGT requests is critical
```

Hiring signal: Can you design a trust graph for M&A that minimizes blast radius? (Answer: one-way trusts, SID filtering, separate forests)

Counterexample Lab: Configurations That Look Correct But Fail

Scenario 1: The "Working" Keytab That Isn't

```
# Configuration:
$ klist -ke /etc/webserver.keytab
Keytab name: FILE:/etc/webserver.keytab
KVNO Principal
---- --------------------------------------------------------------------------
   3 HTTP/webserver.corp.com@CORP.COM (aes256-cts-hmac-sha1-96)

# Test:
$ kinit -kt /etc/webserver.keytab HTTP/webserver.corp.com
# ✓ Success! Ticket obtained.

# Application:
$ systemctl start webserver
# ✗ Auth fails: "Server not found in Kerberos database"
```

Why it fails:

```
# Actual hostname:
$ hostname -f
webserver.prod.corp.com

# Client requests:
HTTP/webserver.prod.corp.com@CORP.COM

# Keytab contains:
```

```
HTTP/webserver.corp.com@CORP.COM

# → Mismatch, server rejects ticket
```

The trap: `kinit -kt` tests **client-side** TGT acquisition. It doesn't test **server-side** ticket acceptance.

Correct test:

```
# Test as client:
$ kvno HTTP/webserver.prod.corp.com
HTTP/webserver.prod.corp.com@CORP.COM: kvno = 3

# Test as server:
$ KRB5_KTNAME=/etc/webserver.keytab klist -k
# Then attempt actual service auth
```

Scenario 2: The Reverse DNS Death Spiral

```
# /etc/krb5.conf
[libdefaults]
    rdns = false  # Disabled reverse DNS
```

```
# Configuration:
$ dig webserver.corp.com +short
10.1.2.3

$ dig -x 10.1.2.3 +short
webserver.prod.corp.com.  # Different hostname!

# Keytab:
HTTP/webserver.corp.com@CORP.COM
```

Expectation: With `rdns = false`, library uses forward DNS name → should work.

Reality:

```
$ curl --negotiate https://10.1.2.3/api
# Auth fails!
```

Why it fails:
When you connect by **IP address**, the library has no choice but to do reverse DNS:

```
// libkrb5 logic:
if (is_ip_address(hostname)) {
    // MUST do reverse DNS to get a hostname
    hostname = reverse_dns(ip);
    // rdns=false doesn't apply here
}
```

The trap: `rdns = false` only applies when you connect with a hostname, not an IP.

Fix: Register both SPNs or use DNS load balancer with stable name.

Scenario 3: The 10-Hour Blindspot

```
# Service code:
def get_data():
    cred = gssapi.Credentials(
        name=gssapi.Name('service/host@REALM'),
        usage='initiate',
        store={'keytab': '/etc/service.keytab'}
    )
    # Use credential...

# Deployment:
$ systemctl start myservice
# Works fine for 10 hours
# Then: "Ticket expired" errors
```

Engineer's fix:

```
# /etc/krb5.conf
[libdefaults]
    ticket_lifetime = 24h  # Increase lifetime!
```

Why this doesn't help:
The service **re-creates credentials on every request** (look at the function). Each call to `gssapi.Credentials()` creates a new 10-hour ticket. The problem isn't ticket lifetime—it's that the engineer thinks it is.

Actual problem:

```
# Credentials are created per-request, not cached
# After 10 hours of uptime, the service is fine
# BUT: If the keytab key is rotated during that time,
# and the service doesn't restart,
# the next credential creation will fail
```

Correct diagnosis: Restart the service after keytab rotation, or implement credential caching in-process.

Multi-Perspective Diagnostic Framework

Perspective 1: OS / Library Implementer

When debugging Kerberos, think about **what the library sees**:

```
# Trace library calls:
$ KRB5_TRACE=/dev/stderr kinit user@REALM
[1234] 1708881234.567890: Getting initial credentials for user@REALM
[1234] 1708881234.567900: Sending request to KDC (1 tries)
[1234] 1708881234.568000: Received answer from KDC (42 bytes)
[1234] 1708881234.568100: Selected etype aes256-cts-hmac-sha1-96
[1234] 1708881234.568200: AS key obtained: aes256-cts-hmac-sha1-96/1234
[1234] 1708881234.568300: Storing credentials to FILE:/tmp/krb5cc_1000
```

Key questions:
- What cache is the library writing to? (`krb5_cc_default()` logic)
- What encryption type did it negotiate? (KDC policy vs. client support)
- Did it actually contact the KDC or use cached credentials?
- What SPN did it request? (DNS resolution in the trace)

Library-level debugging:

```
// What libraries actually do when they fail:
krb5_error_code ret;
ret = krb5_get_init_creds_password(ctx, &creds, princ, password, ...);
if (ret) {
    // Libraries just return error codes, no logging by default
    // Application must explicitly log:
    fprintf(stderr, "Kerberos error: %s\n", krb5_get_error_message(ctx, ret));
```

```
}

// Many applications don't log → silent failures
```

Perspective 2: Platform / Infra Engineer

When designing infrastructure, think about **failure domains**:

Single KDC Failure:

```
Impact: Auth blocked for ~5 seconds (UDP timeout)
Mitigation: DNS SRV with multiple KDCs, TCP port 88 open
```

Clock Skew Across Data Centers:

```
Problem: DC1 at +3 min, DC2 at -3 min → 6 min skew
Impact: Tickets from DC1 rejected by DC2
Mitigation: NTP hierarchy with common stratum-1 source
```

Credential Cache Persistence:

```
# Container restart:
# - FILE:/tmp/krb5cc_* → Lost (tmpfs cleared)
# - KEYRING:persistent:* → Lost (kernel session destroyed)
# - KCM: → Survives if KCM daemon external to container

# Design decision:
# Option A: Keytab-based auth on every container start (stateless)
# Option B: KCM daemon in sidecar container (stateful)
```

Ticket Renewal in Long-Running Jobs:

```
# Spark job running for 48 hours:
spark-submit \
    --principal user@REALM \
    --keytab /etc/user.keytab \
    --conf spark.kerberos.keytab=/etc/user.keytab \
    --conf spark.kerberos.principal=user@REALM
# Spark driver auto-renews tickets every 75% of lifetime
# BUT: Executors don't auto-renew → auth fails after 10 hours
```

```
# Fix: Enable executor-side renewal or use shorter job batches
```

Perspective 3: Security Engineer

Attack Surface Analysis:

Credential Cache Theft:

```
# Risk: Unprivileged user reads another user's cache
$ ls -la /tmp/krb5cc_*
-rw------- 1 alice alice 1234 Feb 4 14:00 /tmp/krb5cc_1000
-rw-r--r-- 1 bob   bob   1234 Feb 4 14:05 /tmp/krb5cc_1001  # ← VULNERABLE

# Attack:
$ cp /tmp/krb5cc_1001 /tmp/stolen
$ KRB5CCNAME=/tmp/stolen klist
# → Impersonate bob until ticket expires
```

Keytab Exfiltration:

```
# Risk: Service keytab with excessive permissions
$ ls -la /etc/*.keytab
-rw-r--r-- 1 root root 1234 /etc/http.keytab  # ← VULNERABLE

# Attack:
$ scp server:/etc/http.keytab ./
$ kinit -kt http.keytab HTTP/server.corp.com
# → Impersonate service indefinitely
```

KDC Impersonation:

```
# Risk: Client doesn't validate KDC certificate (rare, but happens)
# Attack: MITM returns fake TGT
# Impact: Client uses fake TGT → attacker knows session key → decrypt traffic

# Defense: KDC certificate pinning in krb5.conf (rare)
```

Golden Ticket Attack (AD-specific):

```
# If attacker gets krbtgt account hash:
# 1. Forge TGT for any user (including non-existent users)
# 2. Set expiry to 10 years
# 3. Add any group memberships (including Domain Admins)
# → Undetectable unless TGT lifetime anomaly is monitored
```

Security Best Practices:
1. **Credential cache isolation:** Use KEYRING or KCM, not FILE
2. **Keytab permissions:** 600, owned by service user
3. **Short ticket lifetimes:** 10h or less
4. **Monitor cross-realm TGTs:** Unusual trust paths = lateral movement
5. **Disable unconstrained delegation:** Use constrained instead
6. **Service account password rotation:** Automate keytab regeneration

---

Perspective 4: Hiring Manager / Interviewer

What I'm actually testing:

Junior Signal:
- Can recite Kerberos protocol steps
- Knows what TGT stands for
- Has used `kinit` in a lab

Mid Signal:
- Has debugged auth failures in production
- Understands credential cache lifecycle
- Can explain clock skew impact

Senior Signal:
- Has designed multi-realm trust architecture
- Has automated keytab rotation at scale
- Can diagnose silent failures from symptoms alone
- Understands security trade-offs (e.g., delegation types)

Staff Signal:
- Has migrated authentication systems (LDAP → Kerberos, AD → IPA)
- Has written libraries/wrappers around GSSAPI
- Can explain performance impact of AD group bloat on ticket size
- Has designed disaster recovery for KDC failures

Expert-Level Diagnostic Questions

Question 1: The Intermittent Failure

```
Symptom: Service auth succeeds 90% of the time, fails 10% randomly.
Errors: "Clock skew too great" intermittently
Environment: 50-node cluster behind load balancer
```

What's your hypothesis and how do you prove it?

Expected Answer
Hypothesis: Subset of nodes have clock drift. **Diagnostic Steps:**

```
# 1. Check clock skew across all nodes:
for host in node{01..50}; do
    ssh $host "date +%s" &
done | sort | uniq -c
# Look for outliers

# 2. Check NTP sync status:
for host in node{01..50}; do
    ssh $host "chronyc tracking | grep 'System time'"
done
# Identify nodes with >1s offset

# 3. Correlate failures with specific nodes:
# Parse load balancer logs for failure → backend mapping
# Hypothesis: 10% failure rate = 5 out of 50 nodes

# 4. Fix:
# Restart chronyd on affected nodes
# Verify NTP source reachability
```

Deeper diagnosis: Why did only some nodes drift? - VM clock reset after snapshot restore - NTP source unreachable from specific subnet - Leap second handling bug in kernel version

---

Question 2: The Post-Rotation Mystery

```
Sequence of events:
1. Engineer rotates service account password in AD
2. Regenerates keytab: ktpass.exe → new.keytab
```

```
3. Deploys new.keytab to server
4. Service continues working fine
5. 8 hours later: All auth fails with "Decrypt integrity check failed"
```

What happened? What's the fix?

Expected Answer
Root cause: AD retains **both old and new passwords** for a time (password history), but the service is using a **cached TGT from the old password**. **Timeline:**

```
T+0:   Password rotated, new keytab generated (kvno 2)
T+0:   Service has cached TGT from old keytab (kvno 1)
T+0-8h: Service uses cached TGT successfully (AD still accepts it)
T+8h:  TGT expires, service attempts renewal
T+8h:  Renewal attempt uses new keytab (kvno 2)
T+8h:  BUT: AD has purged old password from cache
T+8h:  Service request includes kvno 1 in authenticator
T+8h:  AD tries to decrypt with kvno 1 key → FAILS
```

Fix:

```
# Immediate:
systemctl restart service  # Clears cached TGT, forces new auth with new key

# Long-term:
# 1. Always restart services after keytab rotation
# 2. Or: Use credential caching with explicit refresh
# 3. Or: Automate rotation with zero-downtime (blue/green)
```

Advanced insight: This is why AD has "previous password" support—to allow graceful transitions. But it's time-limited (default: 1 hour).

Question 3: The Load Balancer Kerberos Puzzle

```
Setup:
- VIP: api.corp.com (10.1.2.100)
- Backends: api1.corp.com (10.1.2.101), api2.corp.com (10.1.2.102)
- Keytabs: Each backend has HTTP/api{1,2}.corp.com@REALM

Client auth flow:
1. Client requests ticket for HTTP/api.corp.com
```

```
2. Connects to VIP → routed to api1
3. api1 attempts to decrypt ticket
4. Fails: "Request ticket server does not match principal"
```

Design three different solutions with trade-offs.

Expected Answer
Solution 1: Register VIP SPN on all backends

```
setspn -A HTTP/api.corp.com CORP\api1$
setspn -A HTTP/api.corp.com CORP\api2$
```

Pros: Simple, no client changes **Cons:** Same SPN on multiple accounts (AD allows, but complex ACLs) --- **Solution 2: Use ignore_acceptor_hostname**

```
# /etc/krb5.conf on api1, api2
[libdefaults]
    ignore_acceptor_hostname = true
```

Pros: Backends accept any SPN in ticket **Cons:** Security risk—can't validate which service the ticket was meant for --- **Solution 3: Client-side DNS pinning**

```
# Clients use:
curl --negotiate https://api1.corp.com/api
curl --negotiate https://api2.corp.com/api

# Load balancer does L7 routing based on Host header
```

Pros: Secure, SPN matches backend **Cons:** Clients need service discovery, breaks simple VIP --- **Solution 4: Keytab with multiple SPNs (correct)**

```
# On each backend:
ktpass -princ HTTP/api.corp.com@CORP.COM ...
ktpass -princ HTTP/api1.corp.com@CORP.COM ...
# Merge into single keytab:
ktutil
  rkt api.keytab
  rkt api1.keytab
  wkt /etc/merged.keytab
```

Pros: Supports both VIP and direct access **Cons:** Requires careful keytab management **Best practice:** Solution 4 for production. Solution 1 if AD policies allow.

Question 4: The Ticket Size Death Spiral

```
Symptom: Auth succeeds for some users, fails for others
Error (client): "Message too long"
Error (server): No error, connection reset
Environment: AD with 500+ group memberships per user
```

Explain the failure mechanism and propose a fix.

Expected Answer
Failure mechanism:

```
1. User authenticates, AD issues TGT
2. TGT includes PAC (Privilege Attribute Certificate)
3. PAC contains all group SIDs (500+ groups)
4. TGT size: ~48KB (normal: 4KB)

5. Client requests service ticket
6. Service ticket also includes PAC
7. KDC sends response via UDP (port 88)
8. UDP packet size: ~50KB
9. UDP MTU: 1500 bytes
10. Packet fragmentation → some fragments lost
11. Client timeout, no retry
```

Why it affects some users: Group membership count varies. Power users with many nested groups exceed UDP limits. **Library behavior:**

```
// MIT Kerberos (simplified):
if (response_size > UDP_MAX) {
    // Should retry with TCP, but some old clients don't
    return KRB5_ERR_RESPONSE_TOO_BIG;
}
```

Fixes: **Client-side (force TCP):**

```
# /etc/krb5.conf
[libdefaults]
    udp_preference_limit = 1  # Force TCP for all requests
```

AD-side (reduce PAC size):

```
# Enable SID compression (Server 2012+):
Set-ADUser $user -Replace @{
    'msDS-PrimaryGroupID'=<primary_gid>
}
# Removes default group from PAC (saves ~100 bytes)

# Or: Reduce nested groups (organizational fix)
```

Infrastructure-side:

```
# Ensure TCP/88 is open in firewalls
# Monitor for UDP fragmentation:
tcpdump -i any -n 'udp port 88 and greater 1400'
```

Advanced insight: This is a known AD scalability issue. Azure AD doesn't include full group list in tokens (uses claims-based auth instead).

Question 5: The Container Credential Cache Challenge

```
Setup:
- Kubernetes pod with init container and app container
- Init container: Runs kinit with keytab
- App container: Needs to use those credentials
- Shared volume: /tmp/krb5

Problem: App container can't find credentials
```

Why does this fail and how do you fix it?

Expected Answer
Why it fails:

```
# Init container:
- name: init-kerberos
  image: kerberos-client
  command: ["kinit", "-kt", "/etc/keytab", "svc@REALM"]
  # Runs as UID 0, writes to /tmp/krb5cc_0
  volumeMounts:
  - name: krb5-cache
    mountPath: /tmp

# App container:
- name: app
  image: myapp
  # Runs as UID 1000
  # Looks for /tmp/krb5cc_1000
  # Can't find credentials → fails
```

Fix 1: Explicit cache path

```
# Init container:
command: ["sh", "-c", "kinit -c /tmp/krb5/krb5cc_shared -kt /etc/keytab svc@REALM"]

# App container:
env:
- name: KRB5CCNAME
  value: "FILE:/tmp/krb5/krb5cc_shared"
```

Fix 2: KCM (Kerberos Credentials Manager)

```
# Sidecar KCM daemon:
- name: kcm
  image: heimdal-kcm
  ports:
  - containerPort: 12121

# Both containers:
env:
- name: KRB5CCNAME
  value: "KCM:"
```

Benefit: Credentials stored in KCM daemon, shared across containers. **Fix 3: Service mesh with identity injection**

```
# Use Istio/Linkerd to inject credentials at runtime
# Credentials obtained by sidecar, injected as headers
# App doesn't need to manage Kerberos at all
```

Trade-offs: - Fix 1: Simple, but cache file permissions tricky - Fix 2: Proper, but requires KCM daemon - Fix 3: Modern, but requires service mesh adoption

---

Final Integration: The Complete Debugging Mindset

When faced with a Kerberos failure in production:

1. Establish the failure boundary:

```
# Where in the chain is it failing?
kinit user@REALM              # Client → KDC (TGT acquisition)
kvno service/host@REALM       # Client → KDC (Service ticket acquisition)
curl --negotiate http://host  # Client → Server (Service auth)
```

2. Check the fundamentals:

```
# Time sync:
chronyc tracking

# DNS resolution:
dig service.example.com
dig -x <IP>

# Credential cache:
klist
ls -l $(echo $KRB5CCNAME | sed 's/FILE://')

# Keytab:
klist -ke /etc/service.keytab
```

3. Enable deep visibility:

```
# Library tracing:
export KRB5_TRACE=/tmp/krb5_trace.log

# Packet capture (port 88 UDP/TCP):
```

```
tcpdump -i any -s0 -w /tmp/krb5.pcap port 88

# KDC logs:
# AD: Event Viewer → Security → 4768/4769/4771
# MIT: /var/log/krb5kdc.log
```

4. Form hypotheses based on error patterns:

```
"Clock skew too great"        → Time sync issue
"Server not found"            → SPN mismatch (DNS or keytab)
"Preauthentication failed"    → Wrong password/keytab
"Ticket expired"              → Renewal failure
"Decrypt integrity failed"    → Key version mismatch
"Request ticket server mismatch" → SPN != keytab principal
```

5. Validate your mental model:

```
# What does the library see?
KRB5_TRACE=/dev/stderr <command> 2>&1 | grep -E "SPN|principal|cache"

# What did the KDC actually issue?
klist -e  # Check encryption type, lifetimes, flags

# What's in the ticket?
# Decode base64 ticket from pcap with MIT's krb5 tools
```

This is the mindset that separates engineers who **fix** Kerberos from those who **restart** it.

You now have the implementation-level understanding to debug Kerberos failures that would stump 95% of engineers, and the conceptual framework to design authentication systems that don't break at 3 AM.